# Name:R.K Ayushman Singh

# Roll no : 261100690014

# AWK Pattern Block – LAB Assignment-D1

**Step 1: Create the employee.txt file**

Open your terminal and create a folder:

```
mkdir -p ~/awk-practical
cd ~/awk-practical
```

Create the file:

```
nano employee.txt
```

Enter the following records:

```
101 Alice Security 72000 Active
102 Bob VLSI 65000 Active
103 Carol Security 48000 Inactive
104 David VLSI 82000 Active
105 Eva Networking 55000 Active
106 Frank Security 90000 Active
107 Grace VLSI 45000 Inactive
108 Henry Networking 76000 Active
109 Irene Security 68000 Active
110 John VLSI 78000 Active
111 Kavya Networking 42000 Inactive
112 Liam Security 85000 Active
```

Save using Ctrl + O, Enter, and Ctrl + X.

Check the file:

```
cat employee.txt
```

Important: The file contains five fields:

| Field | Meaning |
|---|---|
| `$1` | Employee ID |
| `$2` | Employee Name |
| `$3` | Department |
| `$4` | Salary |
| `$5` | Status |

AWK uses $1, $2, etc. to access individual fields.

**Part A – Numerical and String Comparisons**

**Q1. Display the name, department, and salary of employees earning more than 70000.**

Command:

```
awk '$4 > 70000 {print $2, $3, $4}' employee.txt
```

Explanation: $4 > 70000 checks whether the salary is greater than 70000. The print statement displays the employee name, department, and salary.

**Q2. Display employees whose salary is between 60000 and 80000, including both limits.**

Command:

```
awk '$4 >= 60000 && $4 <= 80000 {print}' employee.txt
```

Explanation: >= and <= include both boundary values. && means both conditions must be true.

**Q3. Display employees whose employment status is Inactive.**

Command:

```
awk '$5 == "Inactive" {print}' employee.txt
```

Explanation: $5 represents the status field. == checks whether it exactly matches Inactive.

**Q4. Display employees belonging to the VLSI department.**

Command:

```
awk '$3 == "VLSI" {print}' employee.txt
```

Explanation: $3 represents the department. The command displays records where the department is VLSI.

**Part B – Regular-Expression Patterns**

**Q5. Display all records containing the word Security.**

Command:

```
awk '/Security/ {print}' employee.txt
```

Explanation: /Security/ searches for the word Security anywhere in the record.

**Q6. Display employees whose names begin with the letters A, B, or C.**

Command:

```
awk '$2 ~ /^[ABC]/ {print}' employee.txt
```

Explanation:

- $2 selects the employee-name field.
- ~ means matches a regular expression.
- ^ represents the beginning of a string.
- [ABC] matches A, B, or C.

**Q7. Apply the regular expression only to the employee-name field.**

Command:

```
awk '$2 ~ /a/ {print}' employee.txt
```

**Q8. Display employees whose department field does not match Security.**

Command:

```
awk '$3 !~ /Security/ {print}' employee.txt
```

**Part C – Logical Patterns**

**Q9. Display employees who work in the Security department and earn more than 70000.**

Command:

```
awk '$3 == "Security" && $4 > 70000 {print}' employee.txt
```

Explanation: Both conditions must be true: department must be Security and salary must be greater than 70000.

**Q10. Display employees who belong to either the VLSI or Networking department.**

Command:

```
awk '$3 == "VLSI" || $3 == "Networking" {print}' employee.txt
```

Explanation: || means OR. An employee is displayed if either department condition is true.

**Q11. Display employees who work in VLSI, earn more than 70000, and have Active status.**

Command:

```
awk '$3 == "VLSI" && $4 > 70000 && $5 == "Active" {print}' employee.txt
```

**Part D – Conditional Pattern**

**Q12. Department-dependent salary condition**

- If the employee belongs to Security, salary must be greater than 75000.
- Otherwise, salary must be greater than 70000.

Display employee name, department, and salary.

Command:

```
awk '($3 == "Security" && $4 > 75000) || ($3 != "Security" && $4 > 70000) {print $2, $3, $4}' employee.txt
```

**Part E – Grouped Patterns**

**Q13. Display active employees belonging to either Security or VLSI.**

Command:

```
awk '$5 == "Active" && ($3 == "Security" || $3 == "VLSI") {print}' employee.txt
```

**Q14. Display employees satisfying either condition:**

- Security department and salary greater than 80000.
- VLSI department and salary greater than 75000.

Command:

```
awk '($3 == "Security" && $4 > 80000) || ($3 == "VLSI" && $4 > 75000) {print}' employee.txt
```

**Q15. Display active employees satisfying either condition:**

- Security department and salary greater than 70000.
- VLSI department and salary greater than 75000.

Command:

```
awk '$5 == "Active" && (($3 == "Security" && $4 > 70000) || ($3 == "VLSI" && $4 > 75000)) {print}' employee.txt
```

**Part F – Negated Patterns**

**Q16. Display employees whose status is not Active.**

Command:

```
awk '$5 != "Active" {print}' employee.txt
```

**Q17. Display employees who do not belong to either Security or VLSI.**

Command:

```
awk '$3 != "Security" && $3 != "VLSI" {print}' employee.txt
```

**Part G – Range Patterns**

**Q18. Display all records beginning with employee ID 103 and ending with employee ID 108, including both boundary records.**

Command:

```
awk '$1 == 103, $1 == 108 {print}' employee.txt
```

**ing with Irene, including both boundary records.**

Command:

```
awk '$2 == "David", $2 == "Irene" {print}' employee.txt
```

**Part H – Built-In Variables**

**Q20. Display only the first five records. Include the record number in the output.**

Command:

```
awk 'NR <= 5 {print NR, $0}' employee.txt
```

**Q21. Display records numbered 4 through 9 using NR.**

Command:

```
awk 'NR >= 4 && NR <= 9 {print}' employee.txt
```

**Q22. Display records containing exactly five fields. Print the record number and complete record.**

Command:

```
awk 'NF == 5 {print NR, $0}' employee.txt
```

**Q23. Display only odd-numbered records using NR.**

Command:

```
awk 'NR % 2 == 1 {print}' employee.txt
```

**Part I – Arithmetic Conditions**

**Q24. Assume each employee receives a bonus equal to 10% of the salary. Display employees whose calculated bonus exceeds 7000.**

Print employee name, salary, and calculated bonus.

Command:

```
awk '{bonus = $4 * 0.10} bonus > 7000 {print $2, $4, bonus}' employee.txt
```

**Q25. Calculate the revised salary after adding a 12% increment. Display employees whose revised salary exceeds 85000.**

Command:

```
awk '{revised = $4 * 1.12} revised > 85000 {print $2, $4, revised}' employee.txt
```

**Q26. Calculate monthly salary by dividing annual salary by 12. Display employees whose monthly salary exceeds 6000.**

Command:

```
awk '{monthly = $4 / 12} monthly > 6000 {print $2, $4, monthly}' employee.txt
```

**Part J – Integrated Challenge**

**Q27. Generate a report containing active employees satisfying one of these conditions:**

- Security department and salary at least 70000.
- VLSI department and salary at least 75000.
- Networking department and salary at least 70000.

The output must contain employee ID, employee name, department, salary, and status.

Add a report heading using BEGIN and print the total number of selected employees using END.

Command:

```
awk '
BEGIN {
    print "========================================="
    print "       EMPLOYEE SALARY REPORT"
    print "========================================="
    print "ID NAME DEPARTMENT SALARY STATUS"
}
$5 == "Active" && (
    ($3 == "Security" && $4 >= 70000) ||
    ($3 == "VLSI" && $4 >= 75000) ||
    ($3 == "Networking" && $4 >= 70000)
) {
    print $1, $2, $3, $4, $5
    count++
}
END {
    print "========================================="
    print "Total selected employees:", count
}
' employee.txt
```

Output:

```
=========================================
      EMPLOYEE SALARY REPORT
=========================================
ID NAME DEPARTMENT SALARY STATUS
101 Alice Security 72000 Active
104 David VLSI 82000 Active
106 Frank Security 90000 Active
108 Henry Networking 76000 Active
110 John VLSI 78000 Active
112 Liam Security 85000 Active
=========================================
Total selected employees: 6
```

Explanation:

- BEGIN executes before AWK starts reading the file. It prints the report heading.
- $5 == "Active" checks the employee status.
- && ensures the employee is Active.
- || allows employees from any of the three departments to qualify.
- >= includes the specified minimum salary.
- count++ increases the employee count whenever a record matches.
- END executes after all records have been processed and prints the total number of selected employees.